

FRONT END DEVELOPER - MULTI-STEP LOAN APPLICATION FORM

Category: Complex Form Engineering | Validation Architecture | State Persistence

Timeline: 15 Days | AI-Accelerated Innovation & Research Project

Target: Undergraduate / MBA / Engineering Students

TASK:

Build a production-grade, 8+ step multi-step loan application form with real-time validation, conditional field rendering, document upload with preview and compression, e-signature capture, auto-save with resume functionality, and pre-approval summary generation. Handle three loan types (Personal, Home, Business) with divergent field requirements, PAN/Aadhaar verification simulation, address autocomplete, cross-step validation dependencies, and comprehensive E2E test coverage.

PART A: TRAINING MATERIAL & LEARNING RESOURCES

PART B: GAMIFIED SIMULATION BUSINESS CONCEPT

PART C: TECHNICAL IMPLEMENTATION GUIDELINES & CODE STANDARDS

PART D: 15-DAY PROJECT METHODOLOGY & DELIVERABLES

PART E: NO-CODE TOOLS & PLATFORMS GUIDE

PART F: SUBMISSION PROTOCOL

Important Note:

This Project is designed, developed, and administered solely by Zetheta Algorithms Private Limited ("Zetheta") for the purpose of assessing candidates, students, or event participants on behalf of engaging employers, institutes, or event organisers. Completion of this Project does not guarantee employment, academic outcomes, awards, or any other result - all such decisions remain at the sole discretion of the respective engaging party. Zetheta may use AI-assisted tools to evaluate submissions; such outputs are indicative in nature and may be inaccurate or incomplete. By participating, you consent to your performance data, scores, and submissions being shared with relevant engaging parties for assessment and decision-making purposes.

EXECUTIVE SUMMARY

This project positions the intern as the sole Front-End Engineer at LendSwift, a fictional Indian digital lending startup backed by Series A funding, where 45% of loan applications are abandoned mid-flow due to form complexity, confusing validation messages, and the absence of save-and-resume functionality. The intern receives the current broken form flow (a monolithic single-page form with inline validation that fires on every keystroke, no auto-save, and no mobile responsiveness) and must rebuild the entire application experience from scratch.

Form engineering is the most underestimated and most frequently broken aspect of fintech front-end development. Unlike portfolio dashboards or data visualisations that tolerate minor display inconsistencies, loan application forms have zero tolerance for error. A single validation bug can cause regulatory non-compliance. A single lost form state can trigger a customer complaint to the RBI Banking Ombudsman. A single accessibility failure can expose the company to litigation under the Rights of Persons with Disabilities Act, 2016.

The intern must handle 50+ form fields spread across 8 mandatory steps with cross-step validation dependencies (for example, the employment type selected in Step 3 determines which income documentation fields appear in Step 5, and the loan amount requested in Step 2 determines whether a co-applicant section activates in Step 6). They must implement auto-save every 30 seconds using LocalStorage with data encryption, design error recovery flows that handle browser crashes and session timeouts, build document upload with client-side compression and preview, implement an e-signature capture component that produces legally-compliant image output, and generate a pre-approval summary that aggregates data from all 8 steps into a reviewable format before final submission.

An Application Completion Rate metric (targeting 85%+, up from the current 55%) and a QA Stress Test (where the evaluator deliberately attempts to break the form through rapid navigation, partial fills, network disconnection simulation, and invalid input injection) serve as the primary evaluation mechanisms. This is not a tutorial project. This is a production simulation that tests whether the intern can ship code that real users would trust with their financial data.

The recommended technology stack is React 18+ with React Hook Form (or Formik) for form state management, Zod or Yup for schema-based validation, Tailwind CSS for utility-first responsive styling, React Dropzone for file uploads, signature_pad/react-signature-canvas for e-signature capture, and Cypress for end-to-end testing. All code must pass ESLint with Airbnb configuration, maintain 80%+ test coverage, and achieve WCAG 2.1 AA accessibility compliance.

PART A: TRAINING MATERIAL & LEARNING RESOURCES

SECTION A1: WHY FORM ENGINEERING IS THE HARDEST FRONTEND PROBLEM

Most junior developers equate frontend development with building visually appealing interfaces. They focus on animations, transitions, colour palettes, and layout grids. While visual design matters, the highest-stakes frontend work in fintech is form engineering. Every digital lending platform, insurance onboarding system, KYC verification portal, and mutual fund purchase flow is fundamentally a multi-step form with validation. When these forms break, money is lost, customers churn, and regulators intervene.

A1.1 The Cost of Broken Forms

Baymard Institute research across 71 studies spanning 2012–2023 found that the average cart/form abandonment rate in financial services is 75.6%. The top five reasons for abandonment are: (1) the process was too long or complicated (26%), (2) the site wanted the user to create an account (25%), (3) hidden costs or requirements appeared late in the flow (22%), (4) the user did not trust the site with credit card or personal information (18%), and (5) errors or crashes occurred during submission (17%). Four of these five reasons are directly attributable to poor form engineering.

In the Indian context, the problem is amplified. RBI's Digital Lending Guidelines (September 2022) mandate that borrowers must receive a Key Fact Statement (KFS) before loan disbursement, that consent must be explicit and granular, and that data collection must follow the principle of minimality. A form that collects unnecessary fields violates regulatory requirements. A form that bundles multiple consents into a single checkbox violates the explicit consent mandate. A form that does not allow the borrower to review all terms before submission violates the KFS requirement. Form engineering in Indian fintech is therefore not merely a UX concern but a compliance imperative.

A1.2 Controlled vs Uncontrolled Components

React offers two paradigms for form inputs: controlled and uncontrolled components. In controlled components, React state is the single source of truth - every keystroke updates state via `onChange`, and the input value is always derived from state. In uncontrolled components, the DOM itself holds the current value, accessed via refs. The choice between these paradigms has profound implications for form engineering.

Controlled components provide predictability: because state drives the UI, you can implement real-time validation, conditional rendering, and computed fields trivially. However, controlled components in large forms (50+ fields) can cause performance degradation because every keystroke triggers a re-render of the parent component and potentially all sibling fields. This is why libraries like `React Hook Form` exist - they use uncontrolled components internally (via refs) but expose a controlled-like API, achieving the best of both paradigms: predictable form state without per-keystroke re-renders.

For this project, the intern must understand this trade-off deeply. The loan application form has 50+ fields. Using naive controlled components with `useState` for each field would create a re-render cascade that degrades performance on mid-range mobile devices (the primary device category for Indian digital lending users). React Hook Form's `register` function attaches refs to inputs and only triggers re-renders when explicitly needed (e.g., on validation), making it the recommended choice for this project.

A1.3 Schema-Based Validation with Zod/Yup

Validation is the backbone of financial forms. Unlike social media forms where validation is advisory (you can submit a post without a hashtag), financial forms have hard validation requirements. A PAN number must be exactly 10 characters in the format `AAAAA9999A`. An Aadhaar number must be exactly 12 digits and must pass the Verhoeff checksum algorithm. A loan amount for personal loans under RBI guidelines cannot exceed INR 10,00,000 without additional collateral documentation. These are not suggestions; they are gate conditions.

Schema-based validation libraries like `Zod` and `Yup` allow you to define the shape and constraints of your form data as a declarative schema, then validate inputs against that schema. `Zod` is TypeScript-native and provides type inference, meaning the validated output is automatically typed. `Yup` is more mature and integrates directly with `Formik`. Both integrate with `React Hook Form` via resolvers.

The critical concept for this project is cross-step validation dependencies. Traditional single-step validation is stateless: each field is validated independently. Multi-step form validation is stateful: the validity of fields in Step 5 depends on selections made in Step 3. For example, if the applicant selected 'Self-Employed' as employment type in Step 3, then Step 5 must require ITR documents for the last 3 years (not just salary slips). The validation schema must be dynamic, reconstructed based on the current form state. `Zod`'s `.refine()` and `.superRefine()` methods and `Yup`'s `.when()` method enable this conditional validation.

A1.4 Multi-Step Form Architecture Patterns

There are three common architecture patterns for multi-step forms, each with distinct trade-offs:

Pattern 1: Single Form, Conditional Rendering

One `React Hook Form` instance manages all 50+ fields. Steps are rendered conditionally based on a `currentStep` state variable. Pros: all form data is in a single context, cross-step validation is trivial, and auto-save captures the entire form state in one serialisation. Cons: the form schema is monolithic, performance may degrade with deeply nested conditional schemas, and step-level error isolation is harder.

Pattern 2: Step-Per-Form, Shared Context

Each step is a separate React Hook Form instance. A React Context (or Zustand/Jotai store) holds the accumulated data from all completed steps. When the user completes a step, that step's data is merged into the shared context. Pros: each step validates independently, performance is optimised per-step, and step components are reusable. Cons: cross-step validation requires reading from the shared context (not the current form instance), and auto-save must serialise both the current step's form state and the shared context.

Pattern 3: Wizard Component with Step Registry

A Wizard component maintains a step registry (an ordered array of step configurations). Each step configuration defines its component, validation schema, and dependencies. The Wizard orchestrates navigation, validation gating, and progress tracking. This is the most scalable pattern and is recommended for this project because it cleanly separates concerns: the Wizard handles orchestration, each step handles its own UI and local validation, and a central store handles cross-step state.

A1.5 State Persistence and Auto-Save

In digital lending, users frequently start applications on mobile during their commute and complete them on desktop at home. The RBI's Sachet portal data shows that incomplete loan applications are one of the top five consumer complaints against digital lenders. State persistence is therefore a business-critical feature, not a nice-to-have.

The recommended approach for this project is LocalStorage-based persistence with the following architecture: (1) Every 30 seconds, serialise the current form state (all completed steps + current step's dirty fields) to a JSON string. (2) Before writing to LocalStorage, encrypt the JSON string using AES-256 via the Web Crypto API, because loan application data contains PII (Personally Identifiable Information) including PAN, Aadhaar, income, and address. (3) On page load, check LocalStorage for a saved state. If found, decrypt it, validate it against the current schema version (to handle schema migrations), and offer the user a choice: 'Resume your previous application' or 'Start fresh'. (4) On successful final submission, clear the LocalStorage entry. (5) Implement a TTL (Time To Live) of 72 hours - saved states older than 72 hours are automatically purged to comply with data minimisation principles.

A1.6 File Upload Engineering

Document upload in lending applications is not a simple file input. It requires: (a) client-side validation of file type (PDF, JPG, PNG only), file size (max 5MB per file), and file count (max 3 files per document type); (b) client-side compression for images using Canvas API to reduce upload size by 60–80% without visible quality loss; (c) preview generation showing thumbnails for images and a PDF icon with filename for PDFs; (d) drag-and-drop support via React Dropzone; (e) upload progress indication (simulated for this project since there is no backend); and (f) error recovery if the browser crashes mid-upload (the uploaded files should be recoverable from the auto-save state via base64 encoding, though this has storage limits).

The intern must implement all six of these requirements. A naive file input that simply attaches files to form submission is insufficient and will receive minimal marks.

A1.7 E-Signature Capture

Digital lending in India is increasingly moving towards paperless workflows. The Information Technology Act, 2000 (Section 3A) recognises electronic signatures as legally valid. For this project, the intern must implement an e-signature capture component using the `signature_pad` library (or `react-signature-canvas` wrapper). The component must: (a) provide a canvas area where the user can draw their signature using mouse or touch; (b) display a ‘Clear’ button to reset the canvas; (c) export the signature as a PNG image (base64-encoded); (d) validate that a signature has been provided (non-empty canvas check); (e) display the captured signature in the pre-approval summary for review; and (f) be responsive, adjusting canvas size for mobile viewports.

A1.8 Progressive Disclosure and UX Principles

Progressive disclosure is the design principle of showing users only the information and options relevant to their current task, revealing complexity gradually as needed. In multi-step forms, this manifests as: (a) showing only the current step’s fields; (b) within a step, showing conditional fields only when their trigger condition is met (e.g., showing ‘Business Registration Number’ only when loan type is ‘Business’); (c) showing inline validation errors only after the user has interacted with a field (not on initial render); and (d) showing a progress indicator that communicates how many steps remain without overwhelming the user with the total complexity.

Nielsen Norman Group’s research on form design identifies seven principles that the intern must follow: (1) keep forms short by removing unnecessary fields; (2) visually group related fields; (3) present fields in a single column; (4) use inline validation (but only after blur, not on each keystroke); (5) provide clear, specific error messages (not generic ‘Invalid input’); (6) distinguish optional from required fields; and (7) never clear user input on error.

SECTION A2: TECHNOLOGY STACK DEEP DIVE

A2.1 React 18+ and Concurrent Features

React 18 introduced concurrent rendering, which allows React to interrupt a rendering update to handle a more urgent one. For form engineering, the relevant features are: (a) `useTransition`, which marks state updates as non-urgent, preventing form input lag when expensive computations (like cross-step validation) run; (b) `useDeferredValue`, which defers re-rendering of expensive components until the browser is idle; and (c) `Suspense` for lazy-loading step components, reducing the initial bundle size. The intern should demonstrate awareness of these features even if they choose not to use all of them.

A2.2 React Hook Form (RHF) Architecture

React Hook Form is the recommended form library for this project. Its core innovation is isolating re-renders: when a user types in one field, only that field re-renders, not the entire form. RHF achieves this by using uncontrolled components internally and tracking values via refs. Key APIs the intern must master include: `useForm` (the core hook that returns `register`, `handleSubmit`, `formState`, `watch`, `setValue`, `trigger`, and `reset`), `useFormContext` (for sharing form state across deeply nested components without prop drilling), `useFieldArray` (for dynamic field lists such as multiple employment entries), `useWatch` (for watching specific field values to trigger conditional rendering), and the resolver pattern (for integrating Zod or Yup schemas).

A2.3 Zod Schema Design Patterns

Zod is a TypeScript-first schema validation library that provides both runtime validation and compile-time type inference. For this project, the intern must design schemas that handle: (a) per-step validation (each step has its own schema); (b) cross-step dependencies (Step 5's schema is constructed using data from Step 3); (c) conditional fields using `z.discriminatedUnion()` for loan type branching; (d) custom refinements using `.refine()` for business logic (e.g., EMI-to-income ratio must not exceed 50%); and (e) error mapping using `.transform()` to produce user-friendly error messages. The schemas must be composable: a utility function should accept the current form state and return the appropriate schema for the current step.

A2.4 Tailwind CSS for Form Styling

Tailwind CSS is the recommended styling framework because it enables rapid iteration without CSS file management overhead. For form engineering specifically, the intern must leverage: (a) focus-within and focus-visible utilities for accessible focus management; (b) peer modifiers for showing validation states adjacent to inputs; (c) group modifiers for styling label + input + error message as a unit; (d) responsive utilities (sm:, md:, lg:) for mobile-first form layout; and (e) dark: variants for dark mode support (bonus, not required). The form must be usable on a 320px-wide viewport (the smallest common smartphone width in India, corresponding to low-end Jio phones).

A2.5 Cypress for E2E Testing

Cypress is the recommended E2E testing framework because it runs in the same browser as the application, providing deterministic test execution without flaky waits. The intern must write tests that cover: (a) happy path for each loan type (3 tests); (b) validation error display and recovery for each step (8 tests minimum); (c) auto-save and resume across browser close/reopen (1 test); (d) file upload with preview and removal (1 test); (e) e-signature capture and display in summary (1 test); (f) keyboard-only navigation through the entire form (1 test); and (g) deliberate break attempts (rapid step navigation, back-button behaviour, network disconnection simulation). The minimum passing requirement is 15 distinct user journey tests.



SECTION A3: INDIAN REGULATORY CONTEXT FOR DIGITAL LENDING FORMS

A3.1 RBI Digital Lending Guidelines (September 2022)

The Reserve Bank of India's Guidelines on Digital Lending (DL/2022/01) fundamentally changed how lending forms must be designed in India. Key mandates that directly impact form engineering include:

- **Data Minimality:** Only data necessary for the specific lending purpose may be collected. The form must not include fields for social media handles, contact lists, or device information beyond what is needed for fraud prevention.
- **Explicit Consent:** Each consent (data collection, credit bureau check, communication preferences, data sharing with third parties) must be a separate, clearly labelled checkbox. Pre-ticked checkboxes are prohibited.
- **Key Fact Statement (KFS):** Before final submission, the applicant must be shown a comprehensive summary including the loan amount, tenure, interest rate, EMI, total cost of borrowing, and all fees. This maps directly to the pre-approval summary generator the intern must build.
- **Cooling-Off Period Disclosure:** The form must inform the applicant of their right to exit the loan within a specified cooling-off period without penalty.
- **Grievance Redressal:** The form must display the lender's nodal grievance officer details and the RBI Ombudsman escalation path.

A3.2 PAN and Aadhaar Verification

The intern must implement simulated verification for PAN and Aadhaar. While real verification requires API integration with NSDL (for PAN) and UIDAI (for Aadhaar), the simulation must: (a) validate PAN format (5 letters + 4 digits + 1 letter, where the 4th letter indicates entity type: P for individual, C for company, etc.); (b) validate Aadhaar format (12 digits with Verhoeff checksum on the last digit); (c) simulate an API call with a 1.5-second delay and a mock response (success for valid formats, failure for invalid ones); (d) display a verification badge (green tick) next to the field on successful verification; and (e) prevent progression to the next step until both PAN and Aadhaar are 'verified'.

A3.3 Address and PIN Code Validation

Indian addresses present unique challenges for form engineering. PIN codes are 6 digits and map to specific post offices. The intern must implement: (a) PIN code lookup that auto-fills city, state, and post office name (using a static JSON dataset of Indian PIN codes or the India Post API simulation); (b) separate fields for current address and permanent address with a 'Same as current address' checkbox that copies values; (c) address proof document upload linked to the address step; and (d) validation that the state derived from the PIN code matches the state selected in the dropdown (if there is a discrepancy, warn the user).



SECTION A4: INDUSTRY CASE STUDIES - FORM ENGINEERING WINS AND FAILURES

A4.1 Case Study: How CRED Reduced Loan Form Drop-Off by 38%

CRED's personal loan product launched in 2021 with a 9-step application form that had a 68% drop-off rate. Their engineering team identified three primary issues: (1) the income verification step required manual entry of salary details that were already available from bank statement analysis; (2) the address step did not support PIN code auto-fill, causing 23% of users to enter incorrect state-city combinations; and (3) the document upload step had no client-side compression, causing timeout errors on 3G connections.

The redesigned form reduced steps from 9 to 6 by pre-populating fields from existing CRED data, implemented PIN code auto-fill that reduced address entry errors by 89%, and added client-side image compression that reduced upload failures by 67%. The combined effect was a 38% reduction in drop-off and a 42% increase in completed applications. This case study illustrates that form engineering optimisations have direct, measurable business impact.

A4.2 Case Study: Razorpay's KYC Form and Accessibility Compliance

When Razorpay rebuilt their merchant KYC form in 2022, they made accessibility a first-class concern. Their accessibility audit revealed that 100% of their form validation errors were invisible to screen readers because errors were conveyed solely through colour (red text). They implemented ARIA live regions (`aria-live='polite'`) to announce errors to screen readers, added `role='alert'` to error messages, ensured all form inputs had associated labels (not just placeholder text), and implemented keyboard focus management so that on step transition, focus moved to the first input of the new step. These changes not only improved accessibility scores from 42 to 97 on Lighthouse but also improved completion rates for all users by 12%, because the clearer error communication benefited sighted users as well.

A4.3 Case Study: The PhonePe Insurance Form Incident

In 2023, PhonePe's health insurance application form had a critical bug where the 'Back' button on Step 4 cleared all data from Steps 1–3 because the form state was stored in component-level state that was destroyed on unmount. Over 15,000 users lost their partially completed applications over a weekend before the bug was identified. The fix required migrating form state to a persistent store (they chose Zustand with LocalStorage middleware). This incident is referenced in this project to emphasise that state persistence is not optional - it is a production requirement that prevents real financial and reputational damage.

PART B: GAMIFIED SIMULATION BUSINESS CONCEPT

Note on the Simulation Concept: The gamified simulation concept presented in this section is provided as a reference framework. Students may choose to follow this concept or design their own original gamified simulation - with a different name, structure, narrative, or game mechanics - as long as it meets the assessment criteria outlined in Part F.

SECTION B1: GAMIFIED SIMULATION DESIGN

B1.1 Scenario Briefing

You are the newly hired Front-End Engineer at LendSwift, an RBI-registered NBFC (Non-Banking Financial Company) operating as a digital lending platform. LendSwift serves tier-2 and tier-3 Indian cities, where smartphone penetration is high but network connectivity is unreliable (average 4G speed: 8 Mbps with frequent drops to 2G). The company offers three loan products: Personal Loans (up to INR 10 lakh), Home Loans (up to INR 1 crore), and Business Loans (up to INR 50 lakh).

Your CTO has handed you the following data from the last quarter:

Metric	Current	Target	Gap
Application Completion Rate	55%	85%	30% gap
Average Time to Complete	22 minutes	12 minutes	10 min reduction
Mobile Completion Rate	38%	80%	42% gap
Auto-Save Recovery Rate	0% (not implemented)	95%	Feature missing
Accessibility Score (Lighthouse)	34/100	90+/100	56 point gap
E2E Test Coverage	0 tests	15+ journeys	No testing
Validation Errors Reported	340/month	<20/month	320 reduction
Document Upload Failure Rate	28%	<3%	25% reduction

Your mandate: Rebuild the loan application form from scratch. You have 15 working days. Your submission will be evaluated against these metrics using a QA stress test and automated scoring.



ZeTheta
ADD EXPERIENCE TO KNOWLEDGE

SECTION B2: FORM STEPS SPECIFICATION

B2.1 Complete Step Breakdown

STEP 1: Loan Type Selection & Basic Information

Field	Type	Validation	Conditional Logic
Loan Type	Radio (Personal / Home / Business)	Required	Determines fields in Steps 3, 5, 6
Loan Amount	Number Input with INR formatting	Required; Min 50,000; Max varies by type	Personal: max 10L; Home: max 1Cr; Business: max 50L
Loan Tenure	Dropdown (months)	Required; Range depends on loan type	Personal: 12–60m; Home: 60–360m; Business: 12–120m
Loan Purpose	Dropdown (type-specific options)	Required	Options change based on Loan Type
Referral Code	Text (optional)	Alphanumeric, 6–10 chars if provided	None

STEP 2: Personal Information

Field	Type	Validation	Conditional Logic
Full Name (as per PAN)	Text	Required; 2–100 chars; no special chars except space/period	None
Date of Birth	Date Picker	Required; Age 21–65 years	Age affects max tenure
Gender	Radio (Male/Female/Other)	Required	None
Marital Status	Dropdown	Required	If Married, spouse details in Step 6
Father's Name	Text	Required; same rules as Full Name	None
Mother's Name	Text	Required; same rules as Full Name	None
Email	Email input	Required; valid email format	Verification simulation
Mobile Number	Tel input	Required; 10 digits starting with 6–9	OTP simulation
Alternate Mobile	Tel input	Optional; must differ from primary	None

STEP 3: Identity Verification (KYC)

Field	Type	Validation	Conditional Logic
PAN Number	Text (masked input)	Required; AAAAAA9999A format; 4th char validation	Verification simulation with 1.5s delay
Aadhaar Number	Text (masked, show last 4)	Required; 12 digits; Verhoeff checksum	Verification simulation with 1.5s delay
Aadhaar Consent	Checkbox	Required; must be checked	Cannot proceed without explicit consent
Voter ID (optional)	Text	If provided: 3 letters + 7 digits	None
Passport (optional)	Text	If provided: 1 letter + 7 digits	Shown if Home Loan > 50L

STEP 4: Address Information

Field	Type	Validation	Conditional Logic
Current Address Line 1	Text	Required; 5–200 chars	None
Current Address Line 2	Text	Optional	None
PIN Code	Number (6 digits)	Required; auto-fills city/state/PO	Lookup simulation
City	Auto-filled (editable)	Required	Derived from PIN code

Field	Type	Validation	Conditional Logic
State	Auto-filled (editable)	Required	Must match PIN-derived state
Residence Type	Dropdown (Owned/Rented/Company/Family)	Required	If Rented, show rent amount field
Years at Current Address	Number	Required; 0–50	If < 1 year, show previous address section
Same as Permanent Address	Checkbox	N/A	If unchecked, show permanent address fields
Permanent Address (all fields)	Same as current	Required if checkbox unchecked	Conditional section

STEP 5: Employment & Income Details

Field	Type	Validation	Conditional Logic
Employment Type	Radio (Salaried/Self-Employed/Business Owner)	Required	Determines entire sub-form
Company Name (Salaried)	Text + Autocomplete	Required if Salaried	Conditional
Designation (Salaried)	Text	Required if Salaried	Conditional
Monthly Net Salary (Salaried)	Currency input	Required if Salaried; min 15,000	Used for EMI ratio check
Years of Experience	Number	Required; 0–50	None

Field	Type	Validation	Conditional Logic
Business Name (Self-Emp/Business)	Text	Required if not Salaried	Conditional
Business Type (Self-Emp/Business)	Dropdown	Required if not Salaried	Conditional
Annual Turnover (Self-Emp/Business)	Currency input	Required if not Salaried; min 3L	Conditional
Years in Business (Self-Emp/Business)	Number	Required if not Salaried; min 2	Conditional
Monthly Income (Self-Emp)	Currency input	Required if Self-Employed	Used for EMI ratio check
GST Number (Business)	Text	Required if Business Owner; 15-char format	Conditional
Office/Business Address	Full address block	Required if not Salaried	Conditional

STEP 6: Co-Applicant & Guarantor (Conditional)

This entire step is conditional. It appears when: (a) Loan Amount exceeds INR 5,00,000 for Personal Loans; (b) Always for Home Loans; (c) Loan Amount exceeds INR 20,00,000 for Business Loans. The step collects co-applicant details including name, relationship, PAN, income, and consent.

Field	Type	Validation	Conditional Logic
Co-Applicant Required	Auto-determined	System check	Based on loan type and amount
Co-Applicant Name	Text	Required if step active	None
Relationship	Dropdown (Spouse/Parent/Sibling/Business Partner)	Required if step active	None
Co-Applicant PAN	Text	Same as primary PAN rules	Verification simulation
Co-Applicant Income	Currency input	Required if step active	Added to primary for EMI ratio
Co-Applicant Consent	Checkbox + Signature	Required; cannot proceed without	None

STEP 7: Document Upload & E-Signature

Required documents vary by loan type and employment type. The intern must implement conditional document requirements.

Document	Required For	Accepted Formats	Max Size
PAN Card Copy	All loan types	PDF, JPG, PNG	5 MB
Aadhaar Card (Front + Back)	All loan types	PDF, JPG, PNG	5 MB each
Salary Slips (Last 3 months)	Salaried applicants	PDF	5 MB each
Bank Statements (Last 6 months)	All loan types	PDF	10 MB
ITR (Last 2 years)	Self-Employed / Business Owner	PDF	5 MB each
Property Documents	Home Loan only	PDF	10 MB
Business Registration Certificate	Business Loan only	PDF	5 MB
GST Returns (Last 4 quarters)	Business Loan only	PDF	5 MB each
Photograph (Passport size)	All loan types	JPG, PNG	2 MB
E-Signature	All loan types	Captured via canvas	N/A

STEP 8: Review, Consent & Pre-Approval Summary

The final step aggregates all data from Steps 1–7 into a read-only review page. The intern must implement:

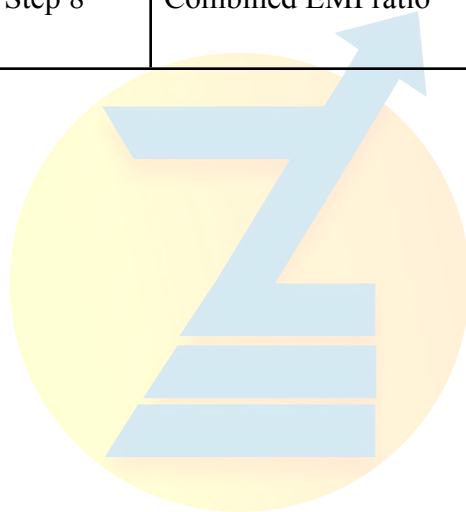
- A section-by-section summary with ‘Edit’ buttons that navigate back to the relevant step.
- A Pre-Approval Summary card showing: Loan Amount, Tenure, Indicative Interest Rate (calculated based on loan type, amount, and applicant profile), Estimated EMI (using the standard EMI formula: $EMI = P \times r \times (1+r)^n / ((1+r)^n - 1)$), Total Cost of Borrowing, and Processing Fee.
- Four separate consent checkboxes: (1) I confirm all information provided is accurate; (2) I authorise LendSwift to check my credit score via CIBIL/Equifax; (3) I agree to the Terms and Conditions (link to PDF); (4) I consent to receive communications regarding this application.
- The e-signature captured in Step 7 displayed for review.
- A ‘Submit Application’ button that is disabled until all four consents are checked and all mandatory documents are uploaded.
- On submission, a success modal with an application reference number (generated client-side as a UUID) and a downloadable PDF summary (bonus feature, not required).

SECTION B3: CROSS-STEP VALIDATION DEPENDENCY MAP

The following table documents all cross-step validation dependencies. The intern must implement every dependency listed. This is the most complex aspect of the project and the primary differentiator between average and exceptional submissions.

Source Step	Source Field	Target Step	Target Field/Behaviour	Rule
Step 1	Loan Type	Step 5	Employment sub-form	Business Loan requires Business Owner or Self-Employed
Step 1	Loan Type	Step 6	Step visibility	Home Loan always shows Step 6
Step 1	Loan Type	Step 7	Document requirements	Different docs per loan type
Step 1	Loan Amount	Step 6	Step visibility	Personal > 5L or Business > 20L triggers Step 6
Step 1	Loan Amount	Step 8	EMI calculation	$EMI = f(\text{amount, tenure, rate})$
Step 1	Loan Tenure	Step 8	EMI calculation	Used in EMI formula
Step 2	Date of Birth	Step 1	Max tenure	Age + tenure must not exceed 65 years
Step 2	Marital Status	Step 6	Co-applicant relationship options	If Married, Spouse is default option
Step 3	PAN verified	Step 7	Upload requirement	If PAN verified, PAN copy upload is optional
Step 4	Residence Type	Step 4	Rent amount field	Rented shows rent amount; used in EMI affordability
Step 5	Employment Type	Step 5	Sub-form fields	Salaried vs Self-Employed vs Business Owner

Source Step	Source Field	Target Step	Target Field/Behaviour	Rule
Step 5	Employment Type	Step 7	Document requirements	Salaried needs salary slips; others need ITR
Step 5	Monthly Income	Step 8	EMI ratio check	EMI must not exceed 50% of income
Step 6	Co-Applicant Income	Step 8	Combined EMI ratio	Combined income used for ratio



ZeTheta
ADD EXPERIENCE TO KNOWLEDGE

SECTION B4: NON-FUNCTIONAL REQUIREMENTS

B4.1 Performance Requirements

- Initial page load (Largest Contentful Paint): < 2.5 seconds on 4G connection.
- Step transition time: < 200ms (no visible jank).
- Validation feedback: < 100ms after field blur.
- Auto-save write: < 50ms (non-blocking, must not freeze the UI).
- Bundle size: < 300KB gzipped for the main chunk (lazy-load step components).
- Memory usage: < 50MB sustained (no memory leaks from file upload previews).

B4.2 Accessibility Requirements (WCAG 2.1 AA)

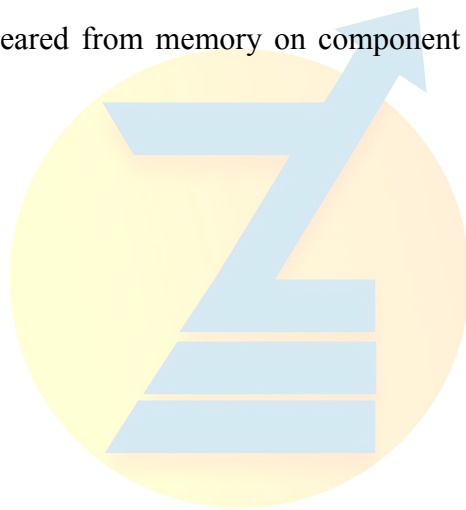
- All form inputs must have associated label elements (not just placeholder text).
- Error messages must use aria-live='polite' and role='alert'.
- Focus must move to the first input of each step on step transition.
- The entire form must be completable using keyboard only (Tab, Shift+Tab, Enter, Space, Arrow keys).
- Colour contrast ratio must be at least 4.5:1 for normal text and 3:1 for large text.
- Touch targets must be at least 44×44 pixels on mobile.
- The progress indicator must convey step position to screen readers via aria-label.
- File upload status must be announced via aria-live region.

B4.3 Browser & Device Support

- Chrome 90+, Firefox 88+, Safari 14+, Edge 90+.
- Responsive: 320px (mobile) to 1920px (desktop).
- Touch and mouse input support.
- Orientation: portrait and landscape on mobile/tablet.

B4.4 Security Requirements

- All PII fields (PAN, Aadhaar, income) must be masked in the UI after entry (show only last 4 characters).
- Auto-save data in LocalStorage must be encrypted using AES-256 via the Web Crypto API.
- The e-signature canvas must not be screen-capture-friendly (implement a basic overlay on blur).
- No PII should appear in console.log statements in production builds.
- Form data must be cleared from memory on component unmount (prevent stale PII in React DevTools).



ZeTheta
ADD EXPERIENCE TO KNOWLEDGE

PART C: TECHNICAL IMPLEMENTATION GUIDELINES & CODE STANDARDS

SECTION C1: PROJECT STRUCTURE

The following directory structure is mandatory. Deviation from this structure will result in a point deduction because the automated assessment tool expects this layout.

SECTION C2: CODE STANDARDS & CONVENTIONS

C2.1 ESLint Configuration

The project must use ESLint with the Airbnb configuration extended with React and accessibility plugins. The following rules are mandatory:

- eslint-config-airbnb: Base Airbnb rules for JavaScript and React.
- eslint-plugin-react-hooks: Enforces Rules of Hooks.
- eslint-plugin-jsx-a11y: Enforces accessibility rules in JSX.
- eslint-plugin-import: Enforces consistent import ordering.
- Zero ESLint errors in production code. Warnings are acceptable only for TODO comments.

C2.2 Component Design Patterns

All reusable form components in the common/ directory must follow these patterns:

- Compound Component Pattern: The Input component must support Input.Label, Input.Field, Input.Error, and Input.HelpText as sub-components.
- Render Props or Children Pattern: The FileUpload component must accept a render function for custom preview rendering.
- Forwarded Refs: All input components must use React.forwardRef to support React Hook Form's register function.
- Controlled + Uncontrolled: Each component must work both as a controlled component (with value/onChange) and as an uncontrolled component (with React Hook Form's register).
- TypeScript Interfaces (Bonus): If using TypeScript, define Props interfaces for every component.

C2.3 Naming Conventions

Entity	Convention	Example
Component files	PascalCase.jsx	Step1LoanType.jsx
Hook files	camelCase.js with 'use' prefix	useAutoSave.js
Schema files	camelCase.js	step1Schema.js
Utility files	camelCase.js	validators.js
CSS classes	Tailwind utilities only	className='flex flex-col gap-4'
State variables	camelCase, descriptive	isStep3Valid, loanTypeSelected
Constants	SCREAMING_SNAKE_CASE	MAX_FILE_SIZE_MB, LOAN_TYPE_PERSONAL
Test files	kebab-case.cy.js	personal-loan-happy-path.cy.js
Git branches	feature/step-N-description	feature/step-3-kyc-verification
Commit messages	Conventional Commits	feat(step3): add PAN verification simulation

C2.4 Git Workflow Requirements

The intern must use a structured Git workflow. The assessment tool will inspect the Git history to verify compliance.

- Main branch: Always deployable. Only merge from feature branches via pull requests (or merge commits with descriptive messages).
- Feature branches: One per step (feature/step-1-loan-type, feature/step-2-personal-info, etc.) plus branches for cross-cutting concerns (feature/auto-save, feature/e2e-tests, feature/accessibility-audit).
- Minimum 40 commits. Submissions with fewer than 40 commits suggest bulk-committing generated code and will be penalised.

-
- Conventional Commit format: type(scope): description. Types: feat, fix, test, refactor, docs, style, chore.
 - No commits larger than 500 lines changed. Large commits suggest AI-generated code dumped without incremental development.
 - README.md must include: project description, setup instructions (npm install, npm run dev, npm run test:e2e), architecture decisions documented, screenshots of each step, and a link to the deployed version (if deployed).



ZeTheta
ADD EXPERIENCE TO KNOWLEDGE

SECTION C3: CRITICAL IMPLEMENTATION DETAILS

C3.1 PAN Validation Algorithm

The PAN (Permanent Account Number) format is: AAAAAA9999A where: positions 1–5 are uppercase letters, positions 6–9 are digits, and position 10 is an uppercase letter. Additionally, the 4th character indicates the entity type: P = Individual, C = Company, H = HUF, A = AOP, B = BOI, G = Government, J = Artificial Juridical Person, L = Local Authority, F = Firm, T = Trust. For this project, only P (Individual) should be accepted for personal and home loans; P or C or F for business loans.

C3.2 Aadhaar Verhoeff Checksum Algorithm

Aadhaar numbers use the Verhoeff algorithm for checksum validation (not Luhn). The intern must implement the Verhoeff checksum using the three lookup tables (multiplication table d, permutation table p, and inverse table inv). The checksum of a valid Aadhaar number is 0 when computed across all 12 digits. This is a non-trivial algorithm and the intern must implement it correctly - not just check for 12 digits.

C3.3 EMI Calculation Formula

The standard reducing-balance EMI formula is: $EMI = P \times r \times (1 + r)^n / ((1 + r)^n - 1)$, where P is the principal loan amount, r is the monthly interest rate (annual rate / 12 / 100), and n is the loan tenure in months. The interest rate is determined by: Personal Loan: 10.5% p.a. (fixed for simulation); Home Loan: 8.5% p.a.; Business Loan: 14% p.a. The Total Cost of Borrowing = $(EMI \times n) - P$. The Processing Fee = 1% of loan amount (minimum INR 2,000, maximum INR 25,000). All values must be displayed with Indian number formatting (e.g., 10,50,000 not 1,050,000).

C3.4 Auto-Save Implementation Specification

The auto-save mechanism must follow this exact specification:

- Create a custom hook useAutoSave(formState, interval = 30000) that accepts the current form state and save interval.
- Use useRef to store a debounced timer. On every form state change, reset the timer.
- When the timer fires, serialise the form state to JSON.
- Encrypt the JSON using AES-256-GCM via window.crypto.subtle. Use a static key derived from a passphrase (for this project, a hardcoded key is acceptable; in production, this would be a user-specific key).

-
- Write the encrypted string to localStorage under the key 'lendswift_draft_[loanType]'.
 - Store a metadata object alongside: { version: '1.0', timestamp: ISO string, step: currentStep, loanType }.
 - Display a subtle toast notification: 'Draft saved at [time]' that auto-dismisses after 2 seconds.
 - On page load, check for a saved draft. If found and less than 72 hours old, show a modal: 'You have a saved application for [Loan Type]. Resume or Start Fresh?'
 - If the user chooses Resume, decrypt the saved state, validate it against the current schema version, and restore the form to the saved step.
 - If the user chooses Start Fresh, or if the saved state fails schema validation, delete the saved state and start from Step 1.



SECTION C4: IMAGE COMPRESSION SPECIFICATION

The intern must implement client-side image compression for uploaded documents. The algorithm must:

- Accept the uploaded File object.
- Create an Image element and load the file via `URL.createObjectURL`.
- Create a Canvas element with the target dimensions (max 1200px width, maintaining aspect ratio).
- Draw the image onto the canvas using `drawImage`.
- Export the canvas as a JPEG blob using `canvas.toBlob` with quality parameter 0.7 (70% quality provides ~60–80% file size reduction with minimal visible quality loss).
- If the resulting blob is still larger than 2MB, recursively reduce quality by 0.1 until it fits or quality reaches 0.3.
- Replace the original file in the form state with the compressed version.
- Display both original and compressed file sizes to the user.

This compression must only apply to image files (JPG, PNG). PDF files must not be compressed client-side. The compression must be non-blocking (use `async/await` with Promises) to prevent UI freezing.

PART D: PROJECT METHODOLOGY AND DELIVERABLES

SECTION D1: OVERVIEW

Sprint	Days	Focus	Key Deliverables
Sprint 1: Foundation	Days 1–5	Project setup, core components, Steps 1–4	Wizard component, 4 steps with validation, PIN code lookup
Sprint 2: Complexity	Days 6–10	Steps 5–8, cross-step logic, auto-save	Conditional rendering, file upload, e-signature, auto-save
Sprint 3: Quality	Days 11–15	Testing, accessibility, polish, documentation	15+ E2E tests, WCAG audit, README, deployment

SECTION D2: DAILY BREAKDOWN

DAY 1: Project Setup & Architecture Foundation

Morning (4 hours):

- Initialise Vite + React project: `npm create vite@latest loan-application -- --template react`.
- Install all dependencies: `react-hook-form`, `@hookform/resolvers`, `zod`, `tailwindcss`, `postcss`, `autoprefixer`, `react-dropzone`, `react-signature-canvas`, `cypress`.
- Configure Tailwind CSS with custom theme colours (LendSwift brand: primary blue #1F4E79, accent green #27AE60, error red #E74C3C, warning amber #F39C12).
- Set up ESLint with Airbnb config + `jsx-a11y` plugin.
- Create the directory structure as specified in Section C1.

Afternoon (4 hours):

- Build the Wizard component skeleton: step registry, `currentStep` state, next/prev navigation, progress calculation.
- Build the ProgressBar component showing step names and completion percentage.
- Build the StepNavigation component with Previous/Next/Save Draft buttons.
- Commit: `feat(setup)`: initialise project with Vite, Tailwind, RHF, Zod

End-of-Day Deliverable:

A running application that displays the Wizard with empty step placeholders and a functional progress bar. Navigation between placeholder steps works. Minimum 3 commits.

DAY 2: Reusable Form Components Library

Morning:

- Build Input component with label, error message, help text, and forwardRef support.
- Build Select component (native and custom dropdown variants).
- Build RadioGroup component with horizontal and vertical layouts.
- Build Checkbox component with linked label.

Afternoon:

- Build CurrencyInput component with INR formatting (Indian number system: 1,00,000).
- Build MaskedInput component for PAN and Aadhaar (showing only last 4 characters).
- Build an ErrorMessage component with aria-live='polite' and role='alert'.
- Write unit tests (Jest or Vitest) for at least 3 components.

End-of-Day Deliverable:

8 reusable form components in `src/components/common/`, each with accessibility attributes, responsive styling, and RHF compatibility. Minimum 4 commits.

DAY 3: Step 1 (Loan Type) & Step 2 (Personal Info)

Morning:

- Implement `step1Schema.js` with Zod: loan type validation, amount ranges per type, tenure ranges per type, purpose dropdown options.
- Build `Step1LoanType.jsx` with radio buttons for loan type, conditional amount/tenure fields, and real-time validation.

Afternoon:

- Implement `step2Schema.js` with Zod: name validation, DOB with age calculation, email format, mobile number (10 digits starting 6–9), alternate mobile must differ.
- Build `Step2PersonalInfo.jsx` with date picker, gender radio, marital status dropdown, and mobile number validation.
- Implement the cross-step dependency: DOB affects max loan tenure ($\text{age} + \text{tenure} \leq 65$ years). Wire this into Step 1's schema dynamically.

End-of-Day Deliverable:

Steps 1 and 2 fully functional with all validations. Loan type selection dynamically adjusts Step 1 constraints. Minimum 4 commits.

DAY 4: Step 3 (KYC) with Verification Simulation**Morning:**

- Implement PAN validation algorithm in utils/validators.js (format check + 4th character entity type check).
- Implement Aadhaar Verhoeff checksum algorithm in utils/validators.js.
- Build the useVerification custom hook: accepts value and type (PAN/Aadhaar), returns { isVerifying, isVerified, error }.

Afternoon:

- Build Step3KYC.jsx with masked inputs for PAN and Aadhaar.
- Implement the verification simulation: on blur, if format is valid, show a spinner for 1.5 seconds, then show a green 'Verified' badge. If format is invalid, show red error immediately.
- Implement the Aadhaar consent checkbox with regulatory text.
- Connect the PAN verification status to Step 7's document requirements (if PAN is verified, PAN card upload becomes optional).

End-of-Day Deliverable:

Step 3 with PAN and Aadhaar verification simulation, consent management, and cross-step dependency to Step 7. Minimum 4 commits.

DAY 5: Step 4 (Address) with PIN Code Lookup

Morning:

- Create `utils/pinCodeData.json` with at least 100 sample Indian PIN codes (covering all 28 states + 8 UTs) with city, state, and post office name.
- Build the `usePinCodeLookup` custom hook: accepts a 6-digit PIN, returns { city, state, postOffice, isLoading, error }.

Afternoon:

- Build `Step4Address.jsx` with current address fields, PIN code auto-fill, residence type conditional fields (rent amount if Rented), years at address conditional section (previous address if < 1 year), and 'Same as permanent address' checkbox with field copying.
- Implement state-PIN code cross-validation: if user edits the auto-filled state, warn if it mismatches the PIN code.
- Sprint 1 review: Run through Steps 1–4 end-to-end, fix any navigation or validation bugs.

End-of-Day Deliverable:

Step 4 with PIN code lookup, conditional rendering, address copying. All 4 steps navigate cleanly. Sprint 1 is complete. Minimum 4 commits.

DAY 6: Step 5 (Employment & Income) - The Hardest Step

Full Day:

- Implement `step5Schema.js` using `z.discriminatedUnion()` for the three employment types (Salaried, Self-Employed, Business Owner). Each branch has entirely different field requirements.
- Build `Step5Employment.jsx` with dynamic sub-forms that mount/unmount based on employment type selection.
- Implement the GST number validation (15-character format: first 2 digits = state code, next 10 = PAN, 13th = entity number, 14th = Z default, 15th = checksum).
- Wire the income field to the EMI affordability check in Step 8 (EMI must not exceed 50% of monthly income).
- If loan type is Business (from Step 1), validate that employment type is Business Owner or Self-Employed (not Salaried).

End-of-Day Deliverable:

Step 5 with all three employment sub-forms, GST validation, income wiring, and cross-step loan type validation. Minimum 3 commits.

DAY 7: Step 6 (Co-Applicant) & Auto-Save Implementation

Morning:

- Build Step6CoApplicant.jsx as a conditional step (visible only when conditions from Section B2 are met).
- Implement the Wizard's step visibility logic: check loan type and amount to determine if Step 6 is active.
- Build co-applicant PAN verification (reuse the useVerification hook) and consent checkbox with signature.

Afternoon:

- Build utils/encryption.js with AES-256-GCM encryption/decryption functions using the Web Crypto API.
- Build hooks/useAutoSave.js following the specification in Section C3.4.
- Build hooks/useFormPersistence.js for the resume/start-fresh modal on page load.
- Test auto-save by filling Steps 1–4, closing the browser, reopening, and verifying the resume modal appears with correct data.

End-of-Day Deliverable:

Step 6 conditional rendering working. Auto-save every 30 seconds with encryption and resume modal. Minimum 4 commits.

DAY 8: Step 7 (Document Upload & E-Signature)

Morning:

- Build the FileUpload component using React Dropzone: drag-and-drop zone, file type validation, size validation, preview generation (thumbnails for images, icon + filename for PDFs).
- Implement image compression using the Canvas API as specified in Section C4.

Afternoon:

- Build the SignatureCanvas component using react-signature-canvas: responsive canvas, clear button, export as base64 PNG, non-empty validation.
- Build Step7Documents.jsx with conditional document requirements based on loan type and employment type (from Steps 1 and 5).

- Display a document checklist with upload status (pending, uploaded, verified).

End-of-Day Deliverable:

Step 7 with conditional document requirements, drag-and-drop upload, image compression, preview, and e-signature capture. Minimum 4 commits.

DAY 9: Step 8 (Review & Pre-Approval Summary)**Morning:**

- Build `utils/emiCalculator.js` with the EMI formula, total cost computation, and processing fee calculation.
- Build the Pre-Approval Summary card with all financial calculations, formatted in Indian number system.

Afternoon:

- Build `Step8Review.jsx` with section-by-section data summary, 'Edit' buttons linking to relevant steps, four consent checkboxes, e-signature display, and the Submit button logic.
- Implement the EMI-to-income ratio validation: if EMI exceeds 50% of monthly income (including co-applicant income if applicable), show a warning but allow submission with additional consent.
- Build the success modal with application reference number (UUID) and application summary.

End-of-Day Deliverable:

Step 8 fully functional with review, calculations, consents, and submission flow. The complete 8-step form is now end-to-end navigable. Minimum 4 commits.

DAY 10: Cross-Step Validation & Polish

Full Day:

- Implement ALL 14 cross-step validation dependencies from Section B3's dependency map.
- Build the schemaFactory.js that accepts the complete form state and returns the appropriate Zod schema for any given step, incorporating all dependencies.
- Test every combination: Personal Loan + Salaried, Personal Loan + Self-Employed, Home Loan + Salaried, Home Loan + Business Owner, Business Loan + Business Owner, Business Loan + Self-Employed.
- Fix all navigation edge cases: back button preserving data, step skipping when Step 6 is inactive, browser back button handling.
- Sprint 2 review: complete walkthrough of all loan types end-to-end.

End-of-Day Deliverable:

All cross-step validations working. Six loan-type/employment-type combinations tested and passing. Sprint 2 is complete. Minimum 4 commits.

DAY 11: E2E Test Suite - Happy Paths

Full Day:

- Write personal-loan-happy-path.cy.js: Complete personal loan application with salaried employment, all valid data, document uploads (use fixture files), e-signature, and successful submission.
- Write home-loan-happy-path.cy.js: Complete home loan application with co-applicant, property documents, and all home-loan-specific fields.
- Write business-loan-happy-path.cy.js: Complete business loan application with GST number, business registration, and ITR uploads.
- Create test fixtures: valid-personal-loan.json, valid-home-loan.json, valid-business-loan.json with complete, valid data for each loan type.
- Create custom Cypress commands in support/commands.js: fillStep1, fillStep2, fillStep3, etc., for reuse across tests.

End-of-Day Deliverable:

3 happy path E2E tests passing. All custom commands created. Minimum 3 commits.

DAY 12: E2E Test Suite - Edge Cases & Validation

Full Day:

- Write `validation-errors.cy.js`: For each step, submit empty, verify all required-field errors appear, verify inline errors appear on blur, verify errors clear on correction. At least 8 test cases (one per step).
- Write `auto-save-resume.cy.js`: Fill Steps 1–4, wait for auto-save, close browser (`cy.clearCookies`, `cy.reload`), verify resume modal appears, choose Resume, verify data is restored.
- Write `file-upload.cy.js`: Upload valid file (verify preview), upload oversized file (verify error), upload wrong type (verify error), upload then remove (verify removal), upload image (verify compression).
- Write `e-signature.cy.js`: Draw on canvas, verify non-empty, clear and verify empty validation, draw again, verify signature appears in Step 8 review.

End-of-Day Deliverable:

12+ E2E tests total (3 happy + 4 edge case suites). All passing. Minimum 3 commits.

DAY 13: E2E Stress Test & Keyboard Navigation

Morning:

- Write `keyboard-navigation.cy.js`: Navigate through entire form using only Tab, Shift+Tab, Enter, Space, and Arrow keys. Verify focus order is logical, all interactive elements are reachable, and form can be submitted without mouse.

Afternoon:

- Write `stress-test.cy.js`: (1) Rapid step navigation: click Next 5 times rapidly, verify no state corruption. (2) Double-submit prevention: click Submit twice rapidly, verify only one submission processes. (3) Back-forward loop: navigate back to Step 1, change loan type, navigate forward, verify conditional steps update correctly. (4) Fill with max-length values in every field, verify no overflow or truncation. (5) Fill with special characters and Unicode in text fields, verify sanitisation.
- Verify total test count is 15+ distinct user journeys.

End-of-Day Deliverable:

15+ E2E tests passing. Keyboard navigation works end-to-end. Stress test passes all scenarios. Minimum 3 commits.

DAY 14: Accessibility Audit & Responsive Polish

Morning:

- Run Lighthouse accessibility audit on every step. Target: 90+ score.
- Install and run axe-core via Cypress (cypress-axe plugin) on every step. Fix all critical and serious violations.
- Verify ARIA labels on all form inputs, error messages, progress bar, and step navigation.
- Verify colour contrast ratios using a contrast checker tool.

Afternoon:

- Test on 320px, 375px, 414px, 768px, 1024px, 1440px, and 1920px viewports. Fix any overflow, truncation, or touch target issues.
- Test in Chrome, Firefox, Safari (via BrowserStack free trial if needed), and Edge.
- Create screenshots of each step at mobile and desktop breakpoints for the README.

End-of-Day Deliverable:

Lighthouse accessibility score 90+ on all steps. All axe-core violations resolved. Responsive layout verified across 7 viewports. Screenshots captured. Minimum 3 commits.

DAY 15: Documentation, Deployment & Final Submission

Morning:

- Write comprehensive README.md: project description, architecture decisions (why Wizard pattern, why RHF over Formik, why Zod over Yup), setup instructions, test running instructions, screenshots, known limitations.
- Deploy to Vercel/Netlify (optional but earns bonus points). Include the deployment URL in README.
- Write ARCHITECTURE.md documenting the Wizard pattern, schema factory, auto-save flow, and cross-step dependency management.

Afternoon:

- Final end-to-end walkthrough of all three loan types.
- Run full Cypress suite one final time. All tests must pass.
- Run ESLint. Zero errors.

- Clean up any TODO comments, console.log statements, and debug code.
- Transfer repository ownership to ZethetaIntern on GitHub.

End-of-Day Deliverable:

Repository transferred to ZethetaIntern. README complete. All tests passing. ESLint clean. Deployed (if applicable). Final commit: chore: prepare for submission.



PART E: TESTING, ACCESSIBILITY & QUALITY ASSURANCE STANDARDS

SECTION E1: E2E TEST REQUIREMENTS

The minimum requirement is 15 distinct E2E test cases covering 15 unique user journeys. The following table lists all required tests. The intern may add additional tests beyond these, which will earn bonus points.

#	Test Name	Description	Priority
1	Personal Loan Happy Path	Complete personal loan, salaried, all valid data, successful submission	P0 (Must Have)
2	Home Loan Happy Path	Complete home loan with co-applicant and property docs	P0
3	Business Loan Happy Path	Complete business loan with GST, ITR, business registration	P0
4	Step 1 Validation Errors	Submit empty Step 1, verify all error messages	P0
5	Step 2 Validation Errors	Submit empty Step 2, verify errors including age validation	P0
6	Step 3 PAN/Aadhaar Validation	Invalid PAN format, invalid Aadhaar checksum, missing consent	P0
7	Step 4 PIN Code Lookup	Enter valid PIN, verify auto-fill; enter invalid PIN, verify error	P0
8	Step 5 Employment Switching	Switch between Salaried/Self-Employed/Business, verify fields update	P0
9	Step 6 Conditional Visibility	Verify Step 6 appears for Home Loan, disappears for small Personal Loan	P1 (Should Have)
10	File Upload & Compression	Upload valid image, verify compression; upload invalid type, verify error	P0

#	Test Name	Description	Priority
11	E-Signature Capture	Draw signature, verify in review; clear, verify validation error	P1
12	Auto-Save & Resume	Fill steps, reload page, verify resume modal, verify data restoration	P0
13	Keyboard Navigation	Complete form using keyboard only (Tab, Enter, Space, Arrows)	P1
14	Rapid Navigation Stress	Click Next/Prev rapidly, verify no state corruption	P1
15	Cross-Step Dependency	Change loan type after reaching Step 5, verify all dependent steps update	P0

P0 tests are mandatory. Failure to implement any P0 test results in an automatic 30-point deduction per missing test. P1 tests are expected. Missing P1 tests result in a 15-point deduction each. Additional P2 (bonus) tests earn up to 30 bonus points total.

SECTION E2: ACCESSIBILITY REQUIREMENTS (WCAG 2.1 AA)

The intern must achieve WCAG 2.1 AA compliance. The following checklist will be used by the QA Stress Test evaluator:

WCAG Criterion	Requirement	How to Verify
1.1.1 Non-text Content	All images have alt text; decorative images have alt=""	axe-core audit
1.3.1 Info and Relationships	All form inputs have associated labels via htmlFor/id	axe-core + manual check
1.3.5 Identify Input Purpose	Input fields use autocomplete attributes (given-name, email, tel, postal-code, etc.)	HTML inspection
1.4.3 Contrast (Minimum)	Text: 4.5:1 ratio; Large text: 3:1 ratio	Lighthouse + contrast checker
1.4.11 Non-text Contrast	Focus indicators and form borders: 3:1 against background	Manual check
2.1.1 Keyboard	All functionality available via keyboard	Cypress keyboard test
2.4.3 Focus Order	Focus order matches visual order; step transitions move focus to first input	Manual + Cypress
2.4.7 Focus Visible	All interactive elements have visible focus ring	Manual check
3.3.1 Error Identification	Errors described in text (not just colour)	Manual + axe-core
3.3.2 Labels or Instructions	Every input has a label; required fields marked with text (not just asterisk)	axe-core

WCAG Criterion	Requirement	How to Verify
3.3.3 Error Suggestion	Error messages suggest correction (e.g., 'PAN must be 10 characters in format AAAAA9999A')	Manual review
4.1.2 Name, Role, Value	Custom components expose role and state to assistive tech via ARIA	axe-core + screen reader



ZeTheta
 ADD EXPERIENCE TO KNOWLEDGE

SECTION E3: QA STRESS TEST PROTOCOL

The QA Stress Test is conducted by the evaluator after submission. The evaluator will attempt to break the form using the following attack vectors. The intern should anticipate these and build defences.

E3.1 Data Integrity Attacks

- Open browser DevTools, navigate to Application > LocalStorage, manually modify the saved form state. On page reload, the form should detect corruption (via schema validation) and offer to start fresh rather than loading corrupted data.
- In Step 5, select Salaried, fill all salaried fields, then go back to Step 5 and switch to Self-Employed. Verify that salaried-specific data is cleared and self-employed fields are empty (no stale data leaks).
- In Step 1, select a Personal Loan with an amount 3,00,000 (below Step 6 threshold). Proceed to Step 5. Then navigate back to Step 1 and change the amount to 8,00,000 (above threshold). Navigate forward. Step 6 must now appear. If it was previously skipped, the Wizard must insert it.

E3.2 Performance Attacks

- Open 3 tabs of the application simultaneously. Fill forms in all 3 tabs. Verify that auto-save does not corrupt data (each tab should have its own draft or warn about conflicts).
- Upload 5 images of 4.5MB each in Step 7. Verify that the compression runs without freezing the UI and that total compressed size is reasonable.
- Rapidly click the Next button 20 times. The form should not skip steps, corrupt state, or throw errors.

E3.3 Accessibility Attacks

- Navigate the entire form using only VoiceOver (macOS) or NVDA (Windows). Every field label, error message, and navigation element must be announced.
- Set browser zoom to 200%. The form must remain usable without horizontal scrolling.
- Set browser font size to 24px (via accessibility settings). The form layout must accommodate the larger text without overflow.

E3.4 Edge Case Attacks

- Enter a PAN number of ABCDE1234F (invalid because 4th character is E, which is not a valid entity type). The form should show a specific error: ‘PAN 4th character must indicate entity type (P for Individual, C for Company, etc.)’, not a generic ‘Invalid PAN’.
- Enter a date of birth that makes the applicant exactly 21 years old today. The form should accept it. Enter a DOB that makes them 20 years and 364 days old. The form should reject it with a clear message.
- Enter a loan amount of exactly INR 5,00,000 for a Personal Loan (the threshold for Step 6). Verify whether Step 6 appears or not (the requirement says ‘exceeds’, so exactly 5,00,000 should NOT trigger Step 6; only 5,00,001+ should).



ZeTheta
ADD EXPERIENCE TO KNOWLEDGE

SECTION E4: PERFORMANCE BENCHMARKS

The following Lighthouse scores are expected:

Metric	Minimum	Target	Excellent
Performance Score	70	85	95+
Accessibility Score	85	90	100
Best Practices Score	80	90	100
SEO Score	70	80	90+
First Contentful Paint	< 2.5s	< 1.8s	< 1.2s
Largest Contentful Paint	< 3.5s	< 2.5s	< 1.5s
Total Blocking Time	< 500ms	< 200ms	< 100ms
Cumulative Layout Shift	< 0.25	< 0.1	< 0.05

PART F: SUBMISSION PROTOCOL

Projects are assessed using a comprehensive 1000-point rubric covering seven dimensions: Problem Understanding, Solution Quality, Research & Analysis, Presentation & Clarity, Innovation & Creativity, Feasibility & Practicality and CV alignment. Assessment is conducted through a combination of manual review and AI-powered evaluation.



END OF PROJECT DOCUMENT